Evidence-Traceable PPA Analysis of 64-bit Array, Radix-4 Booth, and Dadda Multipliers Using an Open-Source SKY130 Flow

Nikhil Ramanathan
Independent Undergraduate VLSI Project
SRM Institute of Science and Technology
Chennai, India

***Abstract*—Multipliers are central arithmetic blocks in processors, digital signal processing units, embedded systems, and accelerator datapaths. This paper presents an evidence-traceable comparison of three 64-bit multiplier architectures: a conventional array multiplier, a Radix-4 Booth multiplier, and a Dadda multiplier. Each architecture was implemented in Verilog/SystemVerilog and evaluated through an open-source ASIC-style flow using Yosys, ABC, the SKY130 high-density standard-cell library, OpenSTA timing analysis, and OpenROAD post-synthesis power estimation. The final reported values are tied to raw evidence files in a public repository rather than only to a result table. The final mapped results show that the Radix-4 Booth multiplier achieves the smallest mapped area of 122,304.800000 library area units, the shortest reported critical path delay of 62.6285 ns, the highest estimated maximum frequency of 15.97 MHz, and the lowest regenerated post-synthesis power estimate of 3.34 mW. The Dadda multiplier achieves the lowest mapped cell count of 12,744 cells. All three current combinational implementations violate a 10 ns virtual clock constraint, so the timing results are comparative critical-path measurements rather than timing-closed implementation claims. The study demonstrates architectural trade-offs among partial-product generation, reduction strategy, mapped area, delay, and power while emphasizing reproducible reporting practices.**

***Index Terms*—ASIC, multiplier architecture, Yosys, OpenSTA, OpenROAD, SKY130, Radix-4 Booth multiplier, Dadda multiplier, array multiplier, PPA analysis, Verilog.**

I. Introduction

Hardware multiplication is a fundamental operation in general-purpose processors, digital signal processing systems, machine-learning accelerators, and embedded computing platforms. Although multiplication is conceptually simple, the hardware cost of wide multipliers grows rapidly with operand width. For a 64-bit multiplier, the number of generated partial products, the structure of the reduction network, and the final accumulation path strongly influence area, timing, and power.

This work compares three representative unsigned 64-bit multiplier architectures. The first is a conventional array multiplier, which has a regular and intuitive structure but a long accumulation path. The second is a Radix-4 Booth multiplier, which reduces the effective number of partial products through multiplier recoding. The third is a Dadda multiplier, which reduces partial products using a height-constrained compressor-style reduction strategy. The project is scoped as an undergraduate-level ASIC front-end and early physical-evaluation study: the focus is on RTL design, synthesis, timing analysis, power estimation, evidence-backed comparison, and clear documentation rather than a full production signoff flow.

A major contribution of this paper is the evidence discipline behind the comparison. Student and portfolio projects often report cell count, area, delay, or power without enough raw evidence to audit the values. In this version, the reported numbers are tied to raw Yosys, OpenSTA, and OpenROAD report files in a public repository. This makes the project more credible for academic review, portfolio evaluation, and future extension.

II. Related Background

A. Array Multiplication

An array multiplier forms partial products through bitwise AND operations and accumulates them in a regular structure. Its main advantage is conceptual simplicity and layout regularity. Its disadvantage is that wide operands can produce long combinational paths and large switching activity, especially when partial-product accumulation is not deeply optimized.

B. Radix-4 Booth Multiplication

Booth multiplication reduces the number of partial products by recoding the multiplier. Radix-4 Booth recoding groups multiplier bits and selects among multiples such as 0, $\pm X$, and $\pm 2X$. This can reduce the number of partial-product rows compared with a direct array approach, but it introduces recoding, sign handling, and correction logic.

C. Dadda Multiplication

Dadda multiplication reduces partial products using a staged reduction method that delays compression until necessary and targets prescribed column-height limits. Compared with a regular array structure, this can reduce the number of reduction operations and lower cell count, depending on implementation and mapping.

III. Design Scope

The evaluated designs are 64-bit unsigned multipliers producing 128-bit products. The top-level RTL files used for the final comparison are listed in Table I.

TABLE I: RTL Designs Used in Final Evaluation

Architecture	RTL file	Top module
Array	`designs/array/design.sv`	`array_multiplier_64bit`
Radix-4 Booth	`designs/booth/design.sv`	`booth_multiplier_64bit`
Dadda	`designs/dadda/design.sv`	`dadda_multiplier_64bit`

The repository also contains archived Booth RTL modules retained for reference. These archived modules are not the final Booth implementation used for the reported PPA comparison. This distinction is important because the reported results must correspond exactly to the RTL that was synthesized and analyzed.

IV. Methodology

A. Area Flow

Area was generated using Yosys and ABC with the SKY130 HD typical Liberty file `sky130_fd_sc_hd__tt_025C_1v80.lib`. For each design, Yosys read the SystemVerilog RTL, elaborated the selected top module, applied standard optimization passes, performed technology mapping with ABC, and reported mapped cell area using `stat -liberty`. The mapped area is reported in Liberty library area units. It should not be interpreted as final post-layout silicon area because no detailed placement, routing, parasitic extraction, or signoff physical verification is claimed in this work.

B. Timing Flow

The mapped Verilog netlists generated by Yosys were analyzed using OpenSTA. A 10 ns virtual clock was applied consistently across all architectures. Input and output delays were set to 0 ns to isolate the combinational input-to-output delay of the multiplier logic. The reported critical path delay corresponds to the data arrival time of the worst reported input-to-output path. The slack values correspond to the same 10 ns virtual clock constraint.

C. Power Flow

Power was regenerated using OpenROAD from the final SKY130-mapped Verilog netlists stored under `results/netlists/`. The reports use a uniform global activity assumption with activity factor 0.10 and duty cycle 0.50. The setup also uses a 10 ns clock period, 0.10 ns input transition, and 0.05 pF output load. The reported values should be interpreted as post-synthesis comparative power estimates, not post-layout signoff power, because placed-and-routed DEF parasitics were not used.

D. Evidence Policy

Only values traceable to raw files are reported in the final comparison. The final evidence files are summarized in Table II. This prevents unsupported result claims and makes the repository auditable.

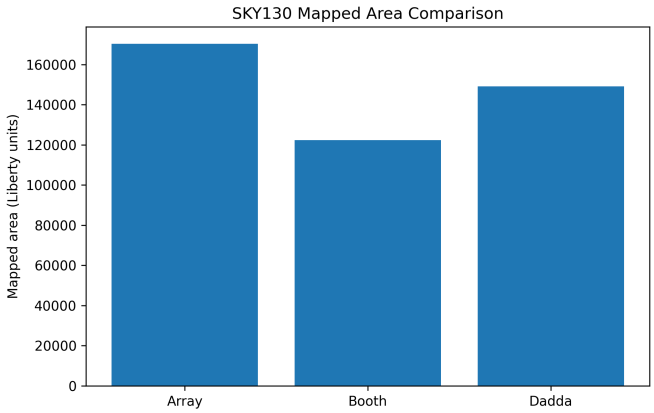

Fig. 1: SKY130 Liberty-mapped area comparison.

V. Code and Evidence Availability

The RTL source files, synthesis scripts, generated SKY130-mapped netlists, raw Yosys area reports, raw OpenSTA timing reports, regenerated OpenROAD power reports, generated figures, and result summaries are available in the public project repository:

https://github.com/NikhilRamanathan/64bit-Multiplier-Architecture

The corrected regenerated power update is represented by repository commit `c2464d9`. The reported area values are derived from the raw Yosys reports in `results/area/raw/`. The reported timing values are derived from the raw OpenSTA reports in `results/timing/raw/`. The reported power values are derived from regenerated OpenROAD reports in `results/power/`, using the final mapped netlists in `results/netlists/`. An evidence package containing raw reports, mapped netlists, scripts, summaries, and repository documentation was generated from the repository to support reproducibility and result traceability.

VI. Results

A. Final PPA Comparison

Table III presents the final evidence-backed PPA results. These values replace all earlier unsupported or stale summary-only values.

B. Area and Cell Count

The Radix-4 Booth multiplier has the smallest mapped area at 122,304.800000 library area units, while the Array multiplier has the largest mapped area at 170,235.769600 library area units. The Dadda multiplier has the lowest mapped cell count at 12,744 cells, even though its mapped area is larger than the Booth design. This difference shows that cell count alone is not sufficient to infer area, since the chosen mapped cells can differ in drive strength, logic function, and Liberty area.

TABLE II: Raw Evidence Files for Final PPA Results

Metric	Array	Radix-4 Booth	Dadda
Area	`results/area/raw/array_area_raw.txt`	`results/area/raw/booth_area_raw.txt`	`results/area/raw/dadda_area_raw.txt`
Timing	`results/timing/raw/array_sta_raw.txt`	`results/timing/raw/booth_sta_raw.txt`	`results/timing/raw/dadda_sta_raw.txt`
Power	`results/power/array_power_report.txt`	`results/power/booth_power_report.txt`	`results/power/dadda_power_report.txt`
Netlist	`results/netlists/array_sky130_netlist.v`	`results/netlists/booth_sky130_netlist.v`	`results/netlists/dadda_sky130_netlist.v`

TABLE III: Final Evidence-Backed PPA Comparison

Architecture	Mapped Cells	Mapped Area	Critical Path Delay (ns)	Estimated Fmax (MHz)	Slack @ 10 ns (ns)	Total Power (mW)
Array	14,670	170,235.769600	89.8478	11.13	-79.8478	4.03
Radix-4 Booth	14,029	122,304.800000	62.6285	15.97	-52.6285	3.34
Dadda	12,744	149,170.566400	71.4395	14.00	-61.4395	3.49

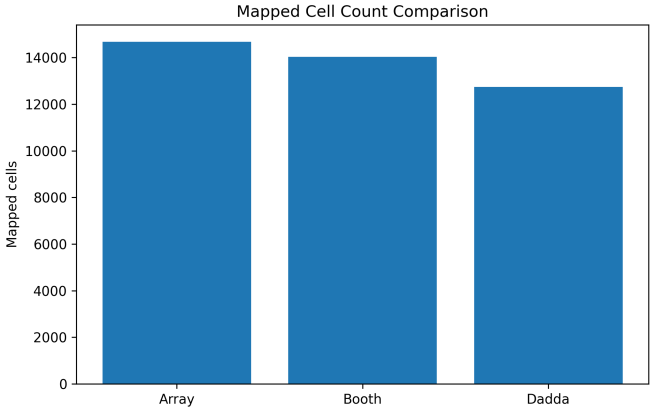

Fig. 2: Mapped cell count comparison.

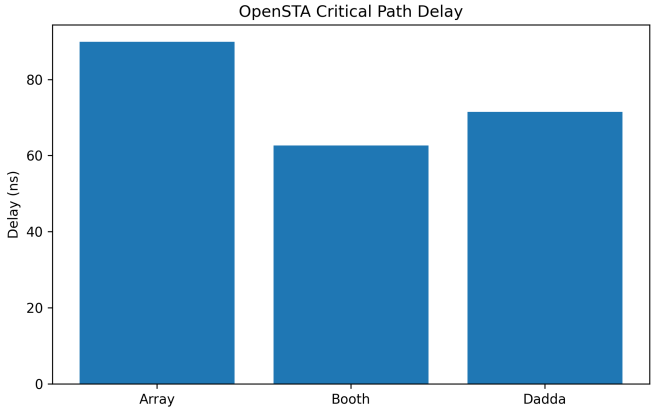

Fig. 3: OpenSTA critical path delay comparison.

C. Timing

The raw OpenSTA reports show that all three current combinational designs violate the 10 ns timing constraint. The Booth multiplier has the shortest reported critical path delay at 62.6285 ns, the highest estimated maximum frequency at 15.97 MHz, and the least negative slack at -52.6285 ns. The Array multiplier has the longest reported critical path delay at 89.8478 ns. The Dadda multiplier reports a delay of 71.4395 ns, which is faster than the Array design but slower than the Booth design in this mapped implementation.

These results should not be interpreted as timing closure. Instead, they provide a comparative view of the longest combinational paths under a shared constraint and library setup. A practical timing-closed design would likely require pipelining, architecture-level retiming, or further logic restructuring.

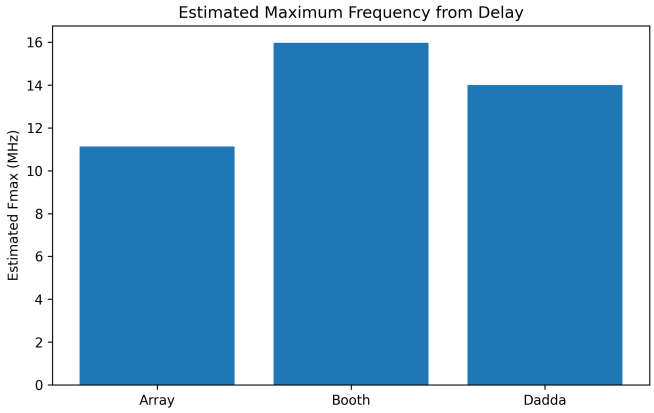

Fig. 4: Estimated maximum frequency from the reported critical path delay.

D. Power

The regenerated OpenROAD reports show that the Radix-4 Booth multiplier has the lowest post-synthesis power estimate at 3.34 mW, followed by the Dadda multiplier at 3.49 mW and the Array multiplier at 4.03 mW. These values are much lower than earlier stale power values because the power reports were regenerated from the final SKY130-mapped netlists using a uniform activity setup. Since no placed-and-routed DEF or extracted parasitics are used, the power values are comparative post-synthesis estimates rather than post-layout signoff measurements.

VII. Discussion

The final evidence-backed results show that the Booth multiplier is the strongest overall design in this particular implementation and flow. It has the smallest mapped area, shortest critical path delay, highest estimated maximum frequency, and

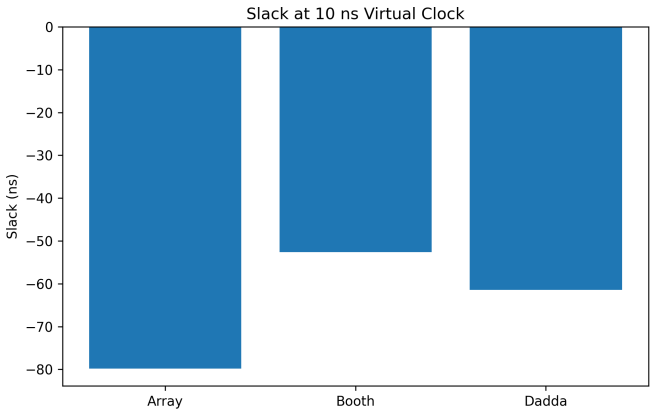

Fig. 5: Slack under a 10 ns virtual clock constraint.

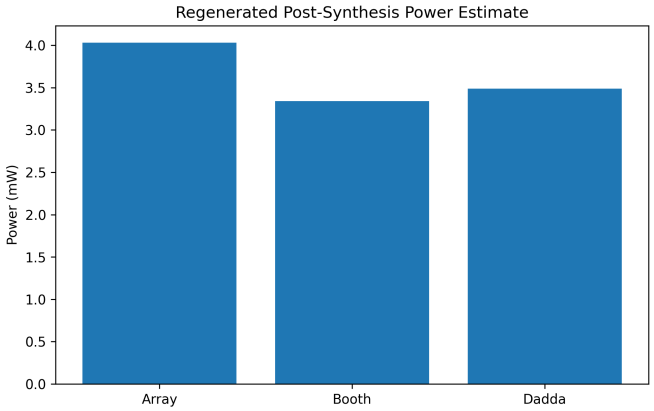

Fig. 6: Regenerated post-synthesis total power estimate.

lowest regenerated post-synthesis power estimate. This result is consistent with the benefit of reducing partial-product rows through recoding, even though the design introduces additional recoding and correction logic.

The Dadda multiplier remains important because it achieves the lowest mapped cell count. However, its lower cell count does not translate to the smallest mapped area or shortest delay in this implementation. The Array multiplier is the least efficient overall, with the largest mapped area, longest critical path delay, and highest regenerated post-synthesis power estimate.

A key lesson from this project is that architectural intuition must be verified through raw tool evidence. A Dadda multiplier is often expected to be fast because of its efficient reduction network. However, in this specific RTL and mapping flow, the Booth implementation reports the shortest critical path. Similarly, mapped cell count, mapped area, delay, and estimated power do not necessarily rank architectures identically. This reinforces the importance of using consistent flow scripts, raw logs, and clearly defined metrics.

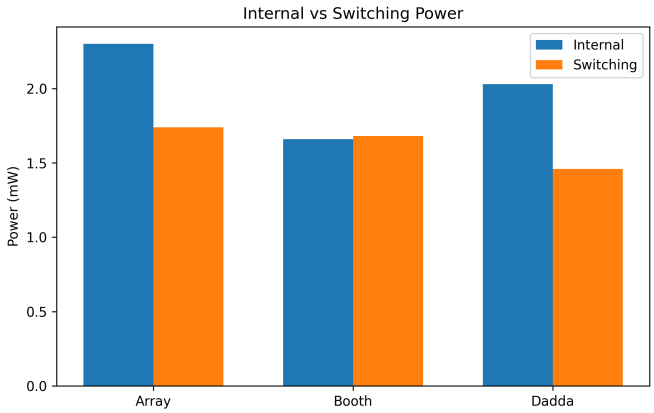

Fig. 7: Internal and switching power components from regenerated OpenROAD reports.

VIII. Limitations and Future Work

This work has several limitations. First, the reported area is based on Liberty-mapped synthesis area, not final placed-and-routed silicon area. Second, the timing analysis uses ideal clocking and zero input/output delay assumptions to compare combinational paths. Third, the current designs violate the 10 ns timing target, so the results do not demonstrate timing closure. Fourth, the power results are post-synthesis estimates generated from mapped netlists under uniform global activity assumptions; they are not parasitic-aware post-layout signoff power values.

Future work should include pipelined versions of all three architectures, consistent testbench-driven switching activity generation for each design, OpenROAD placement-and-routing, parasitic-aware timing analysis, and comparison under multiple clock periods. Additional future extensions could include signed multiplication support, compressor-tree variants, Wallace reduction, and FPGA implementation for hardware demonstration.

IX. Conclusion

This paper presented an evidence-traceable PPA comparison of 64-bit Array, Radix-4 Booth, and Dadda multipliers using an open-source SKY130 evaluation flow. The final raw evidence shows that the Booth multiplier achieves the smallest mapped area, shortest reported critical path delay, highest estimated maximum frequency, and lowest regenerated post-synthesis power estimate. The Dadda multiplier achieves the lowest mapped cell count. The Array multiplier is the least efficient across the evaluated metrics. All three designs violate the 10 ns timing constraint in their current combinational form, making the results comparative rather than timing-closed. The final contribution is both technical and methodological: the project demonstrates multiplier architecture trade-offs while maintaining traceability between reported values and raw evidence files.

AI-Assisted Preparation Disclosure

The author used ChatGPT for writing assistance in drafting portions of this manuscript, language editing, organization, and formatting support. All technical content, RTL implementation, simulation, synthesis, timing analysis, power analysis, repository preparation, results, and conclusions are the author's own work and responsibility. The author reviewed and verified the manuscript and remains fully responsible for the accuracy, integrity, and originality of the work.

References

[1] A. D. Booth, "A signed binary multiplication technique," *Quarterly Journal of Mechanics and Applied Mathematics*, vol. 4, no. 2, pp. 236–240, 1951.

[2] C. S. Wallace, "A suggestion for a fast multiplier," *IEEE Transactions on Electronic Computers*, vol. EC-13, no. 1, pp. 14–17, 1964.

[3] L. Dadda, "Some schemes for parallel multipliers," *Alta Frequenza*, vol. 34, pp. 349–356, 1965.

[4] C. Wolf, "Yosys Open SYnthesis Suite," project documentation and source repository. [Online]. Available: https://yosyshq.net/yosys/

[5] R. K. Brayton and A. Mishchenko, "ABC: An academic industrial-strength verification tool," in *Proc. CAV*, 2010, pp. 24–40.

[6] The OpenROAD Project, "OpenSTA static timing analyzer," project repository. [Online]. Available: https://github.com/The-OpenROAD-Project/OpenSTA

[7] The OpenROAD Project, "OpenROAD application," project repository. [Online]. Available: https://github.com/The-OpenROAD-Project/OpenROAD

[8] Google and SkyWater Technology, "SKY130 open-source process design kit," project repository. [Online]. Available: https://github.com/google/skywater-pdk